DATA305 / DATA475 · Machine Learning Methods

From Perceptron to MLP

Architecture · Activation Functions · Keras Sequential API

Lecture 2 Week 5 · Google Colab ready

Binh Nguyen · School of Mathematics and Statistics · Victoria University of Wellington

Today's Agenda

Lecture 2

Six topics

01 **From Perceptron to MLP**

Perceptron limitations · hidden layers · why depth helps

02 **Forward Pass — How an MLP Computes**

Weights, biases, layer-by-layer computation, numeric example

03 **Activation Functions**

Sigmoid · Tanh · ReLU · Softmax — formulas, intuition, defaults

04 **Choosing the Right Loss Function**

MSE for regression · Cross-entropy for classification

05 **Keras Sequential API**

build → compile(loss, optimizer, metrics) → fit → evaluate

06 **Fashion MNIST Demo & Learning Curves**

Training results · reading accuracy curves · overfitting quiz

Lecture 1 → Lecture 2

In Lecture 1 we saw the full DL pipeline at a high level. **Today** we zoom into the **Model** step — building the simplest useful deep model — and the **Train** step — the Keras workflow that makes it run.

Lecture 1 — Covered

- What deep learning is (and is not)
- Gallery of real-world applications
- Classical ML vs. deep learning
- The universal DL pipeline: Data → Model → Train → Evaluate → Iterate
- Live demo: MNIST in ~20 lines with Keras

Lecture 2 — Today

- Perceptron → MLP: why depth enables nonlinear learning
- Forward pass: layer-by-layer numeric computation
- Activation functions: Sigmoid, Tanh, ReLU, Softmax
- Loss functions: MSE, Binary CE, Sparse Categorical CE
- Keras Sequential API — build, compile, fit, evaluate
- Fashion MNIST demo and reading learning curves

From Perceptron to MLP

Biological Neuron (analogy only)

- Receives signals via dendrites
- Cell body sums inputs
- Fires output along axon if threshold exceeded
- ⚠ DL is a mathematical model, not a brain simulation

Perceptron (Rosenblatt, 1958)

- Single layer: $z = w \cdot x + b$
- Step / sigmoid activation
- One linear decision boundary
- Cannot solve XOR

MLP — Multi-Layer Perceptron

- One or more hidden layers
- Each layer: $z = Wx + b$, then activation
- Learn nonlinear feature combinations
- Practical baseline neural network

Key insight: Perceptron = linear only. MLP = hidden layers + nonlinear activation → can learn curved boundaries.

Why Hidden Layers Matter — the XOR Problem

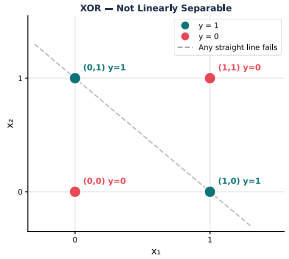
XOR Truth Table

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Not linearly separable!

No single straight line can correctly classify all 4 points.

A Perceptron CANNOT learn XOR.



What Happens with a Hidden Layer?

1. Hidden layer combines inputs into new features
2. Nonlinear activation bends the feature space
3. Output layer can now draw a linear boundary in the transformed space

Result: MLP solves XOR with just 2 hidden neurons.

Deeper layers can combine simple features into richer ones.

For images:

edges → textures → parts → objects

Perceptron vs MLP

	Perceptron	MLP
Layers	1	2+
Boundary	Linear	Nonlinear
Activation	Step / σ	ReLU / etc.
Learns XOR	✗ No	✓ Yes
Use today	No	Yes

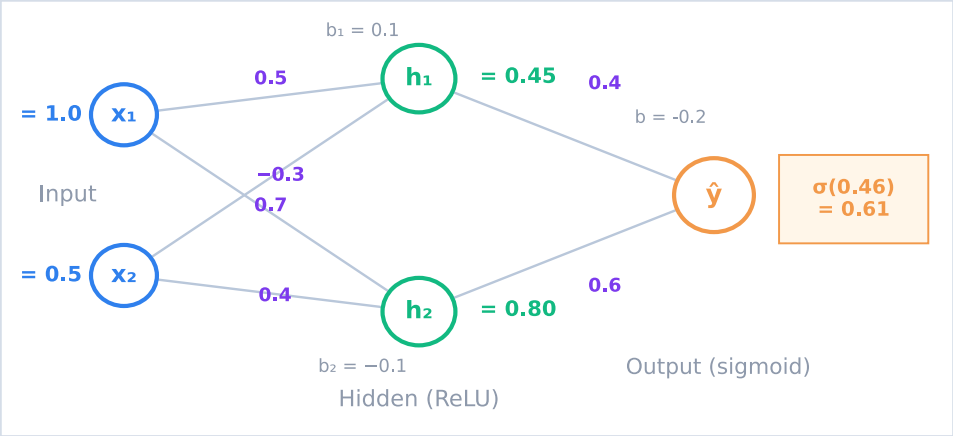
The Forward Pass — Computation Graph


```
z = Wx + b    then    a = ReLU(z)    |    ŷ = σ(z_out)
```

$z_1 = 0.5\times1.0 + (-0.3)\times0.5 + 0.1 = 0.45 \rightarrow \mathrm{ReLU}(0.45) = h_1 = 0.45$

$z_2 = 0.7\times1.0 + 0.4\times0.5 - 0.1 = 0.80 \rightarrow \mathrm{ReLU}(0.80) = h_2 = 0.80$

$z_out = 0.4\times0.45 + 0.6\times0.80 - 0.2 = 0.46 \rightarrow \sigma(0.46) = 0.61$



Example above: 2 inputs → 2 hidden neurons (ReLU) → 1 output (sigmoid) | Network predicts ŷ = 0.61

Core Equations

$$z = Wx + b$$

$$a = f(z)$$

Post-activation (nonlinear step)

Each neuron:
compute weighted sum + bias,
then apply activation function.

Backpropagation — Intuition Only

The key question: how much does each weight contribute to the error — and in which direction should we nudge it?

Forward Pass

Feed inputs x → compute prediction $\hat{y} = f(x; W, b)$.

Compute Loss

$L = \text{loss}(\hat{y}, y)$ — measures how wrong the prediction is.

Backward Pass (Backprop)

Chain rule: compute $\partial L/\partial W$ for every weight; gradients flow from output → input.

Weight Update (Gradient Descent)

```
W = W  −  η · ∂L/∂W   (for every weight in the network)

η   = learning rate   (step size)
∂L/∂W = gradient      (which direction increases loss)
```

Keras / TensorFlow computes all gradients automatically via automatic differentiation.

QUICK QUIZ

A hidden neuron has weights $w = [-0.8, -1.2, -0.5]$ and bias $b = 0.1$.

For input $x = [1.0, 1.0, 1.0]$, its pre-activation is $z = -0.8 - 1.2 - 0.5 + 0.1 = -2.4$.

The layer uses ReLU activation. What does this neuron contribute to the next layer?

A It outputs −2.4, which is passed to the next layer

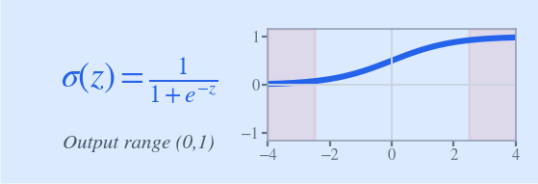
B It outputs 0, because ReLU clips negative values to zero

C It outputs 2.4, because ReLU takes the absolute value

D It outputs 0.08, because ReLU applies a sigmoid to negative values

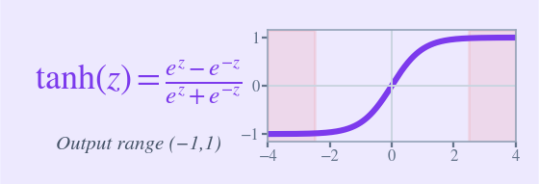
Activation Functions — Sigmoid, Tanh, ReLU

Sigmoid σ(z)



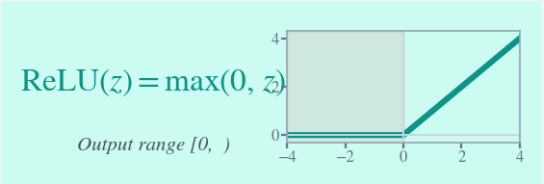
- Output: (0, 1) — interpretable as probability
- Smooth gradient but saturates for large |z|
- Vanishing gradient problem in deep networks
- Use for: output layer, binary classification

Tanh tanh(z)



- Output: (−1, 1) — zero-centered (better than σ)
- Still saturates for large |z|
- Usually better than sigmoid in hidden layers
- Use for: RNNs (we will see in L12)

ReLU max(0, z) Rectified Linear Unit



- Output: 0 or z — non-saturating in positive region
- Gradient = 1 for z > 0: helps gradients propagate more effectively in active regions
- Risk: dead neurons if z stays consistently negative
- Default choice for hidden layers — use this first

Quick guide: Hidden layers → ReLU | Binary output → Sigmoid | Multi-class output → Softmax

Softmax — The Output Layer Activation for Multi-Class

Softmax Formula

$$\text{Softmax}(z_i) = e^{z_i} / \Sigma_j e^{z_j}$$

What it does

Converts K raw scores (logits) into K probabilities

Outputs always sum to exactly 1.0

Highest score → highest probability → predicted class

Use Softmax when…

The task has 3+ mutually exclusive classes (dog vs cat vs fish).
Put Dense(K, activation='softmax') as the final layer.

Combine with…

sparse_categorical_crossentropy (integer labels)
or categorical_crossentropy (one-hot labels).
Do NOT combine with sigmoid.

Binary classification

Use sigmoid (not Softmax) for binary tasks. Softmax on 2 outputs gives correct probabilities but sigmoid is simpler and standard.

Why ReLU Won — Gradient Flow and the Vanishing Problem

The Problem with Sigmoid/Tanh

Sigmoid saturates: for $|z| > 3$, gradient ≈ 0

Backprop multiplies gradients layer by layer

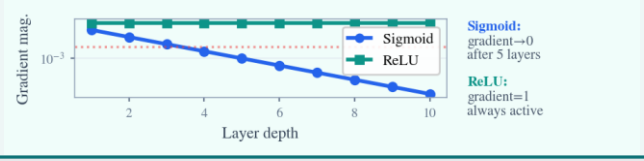
Many small gradients → product approaches zero

Early layers learn very slowly !!!

ReLU Solution

ReLU gradient = 1 when $z > 0$ (in active region)

Helps gradients propagate more effectively in active regions



Practical Decision Guide

Layer type	Activation	Why	Preview
Hidden layer (default)	ReLU	Fast, no saturation in active region	Use He initialization (L4)
Hidden layer (RNNs)	Tanh	Zero-centered, bounded output	L12–L13
Output — binary classif.	Sigmoid	Gives probability in (0,1)	Binary CE loss
Output — multi-class	Softmax	Probabilities summing to 1	Sparse Cat CE loss

QUICK QUIZ

You are building a neural network to classify emails into 5 categories.

The final Dense layer has 5 units. Which activation function should you use in the output layer, and which loss function pairs with it?

- **A** ReLU activation + MSE loss — simple and fast
- **B** Sigmoid activation + Binary cross-entropy — works for any task
- **C** Softmax activation + sparse_categorical_crossentropy — gives probabilities over 5 classes
- **D** Tanh activation + categorical_crossentropy — tanh is zero-centered so more stable

Loss Functions — Matching Task to Loss

Task	Output Layer	Loss Function	Keras string	Example
Regression	Dense(1) — linear	MSE (Mean Squared Error)	`'mse'`	Predict house price, temperature
Binary classification	Dense(1, 'sigmoid')	Binary Cross-Entropy	`'binary_crossentropy'`	Spam vs not-spam, disease vs healthy
Multi-class classif.	Dense(K, 'softmax')	Sparse Categorical CE	`'sparse_categorical_crossentropy'`	MNIST, Fashion MNIST (10 classes)

MSE

$$L = (1/n)\ \Sigma(y_i - \hat{y}_i)^2$$

Penalizes large errors quadratically.

Differentiable everywhere.

Binary CE

$$L = -[y \log \hat{y} + (1-y) \log(1-\hat{y})]$$

Pushes predicted probability toward 0 or 1 to match true label.

Sparse CE

$$L = -\log \hat{y}_c$$

c = true class index.

'Sparse' = labels are integers, not one-hot.

QUICK QUIZ

Match each task to the correct output activation + loss pair:

Task	Correct pairing?
Predict tomorrow's maximum temperature (°C)	(a) linear + MSE (b) sigmoid + BCE (c) softmax + CE
Classify a user review as positive or negative	(a) linear + MSE (b) sigmoid + BCE (c) softmax + CE
Classify a hand-written digit as 0–9	(a) linear + MSE (b) sigmoid + BCE (c) softmax + CE

ANSWER

Answers

Predict temperature (°C)

→ **(a) linear output + MSE**

Continuous target → MSE. Predicted value can be any real number.

Positive/negative review

→ **(b) sigmoid output + binary_crossentropy**

Binary task (2 classes) → sigmoid gives P(positive). BCE is the correct loss.

Digit 0–9

→ **(c) softmax output + sparse_categorical_CE**

10 mutually exclusive classes → softmax gives probabilities over all 10.

Keras Sequential Workflow — Build · Compile · Fit · Evaluate

Build

```
model = Sequential([
  Flatten((28,28)),
  Dense(128,
   activation='relu'),
  Dense(10,
   activation='softmax')
])
```

Define layers in order. Flatten converts 2D image → 1D vector. Specify input shape on first layer only.

→

Compile

```
model.compile(
  optimizer='adam',
  loss='sparse_
   categorical_
   crossentropy',
  metrics=['accuracy'])
```

Specify optimizer, loss, and metrics. Does NOT run any computation yet.

→

Fit

```
history = model.fit(
  x_train,
  y_train,
  epochs=10,
  batch_size=32,
  validation_split=0.1)
```

Train on data for N epochs. validation_split holds out 10% to monitor overfitting.

→

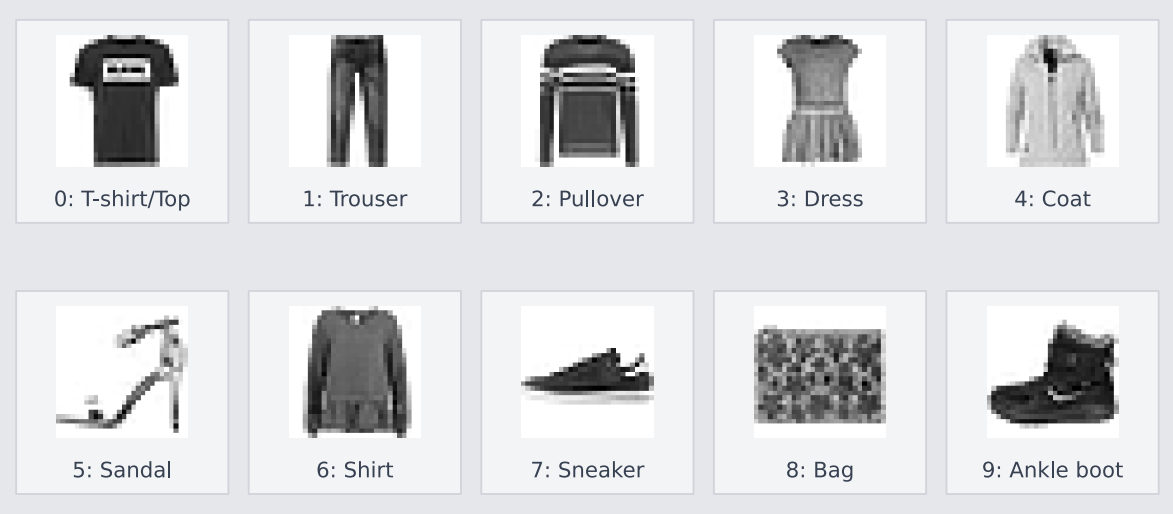
Evaluate

```
loss, acc =
 model.evaluate(
  x_test,
  y_test)
print(f'Test acc:
  {acc:.3f}')
```

Compute loss and metrics on the test set.

Note: Sequential API is ideal for simple stacks of layers. Lecture 3 introduces the Functional API for more complex architectures (multiple inputs/outputs, skip connections).

Dataset — Fashion MNIST



Fashion MNIST — 10 clothing categories, 60 000 training images, 28×28 px

60 000
Training images

10 000
Test images

28 × 28 px
Image size

10 classes
Clothing categories

Why Fashion MNIST instead of MNIST?

MNIST digits are too easy — a simple fully-connected model can reach 99%+ accuracy without any tuning.

Fashion MNIST is harder and more representative of real image classification challenges. Same size and format: drop-in replacement. Loads via tf.keras.datasets.fashion_mnist.load_data().

Fashion MNIST — MLP Classifier in Keras

python · Google Colab — run this now

```python
import tensorflow as tf

# 1. Load & normalize
(x_tr, y_tr), (x_te, y_te) = \
    tf.keras.datasets.fashion_mnist.load_data()
x_tr, x_te = x_tr/255.0, x_te/255.0

# 2. Build
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28,28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10,  activation='softmax')])

# 3. Compile
model.compile(optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])

# 4. Train
history = model.fit(x_tr, y_tr, epochs=10,
    validation_split=0.1)

# 5. Evaluate
loss, acc = model.evaluate(x_te, y_te)
print(f'Test accuracy: {acc:.4f}')
```

Load Data

keras.datasets downloads and caches Fashion MNIST automatically. No manual download needed.

Normalize

Dividing by 255 scales pixel values from [0,255] to [0.0,1.0]. Always normalize inputs before training.

Architecture

Flatten: 28×28 → 784. Dense(128,relu): hidden layer. Dense(10,softmax): 10 class probabilities.

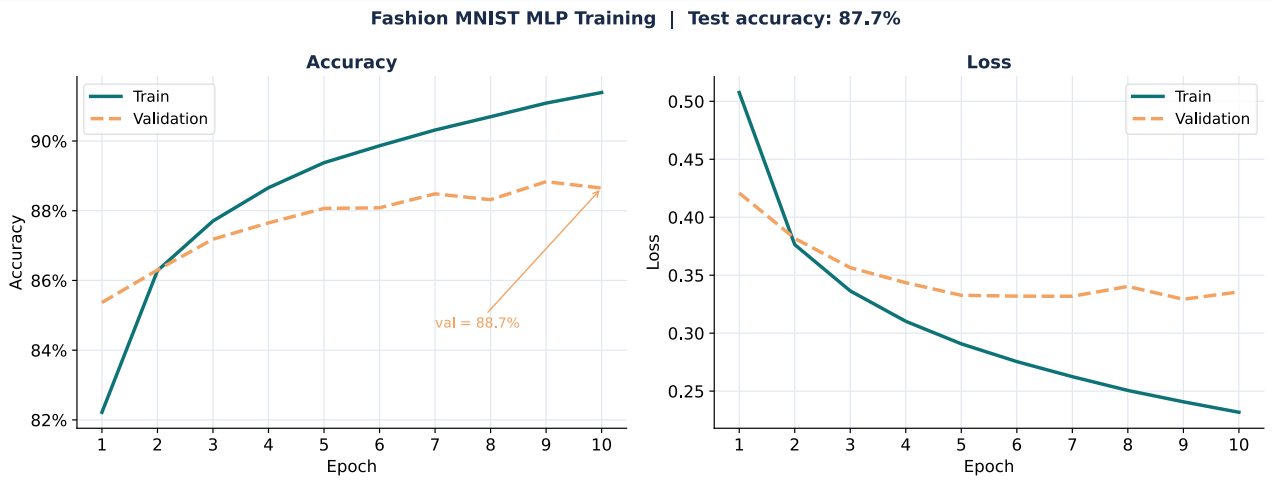
Compile

adam: adaptive optimizer (good default). sparse_categorical_crossentropy: multi-class loss for integer labels.

Fit & Evaluate

validation_split=0.1 holds out 10% of training data to monitor overfitting each epoch.

Fashion MNIST — Training Results



~88 %

Typical test accuracy

few min

Training time on Colab GPU

10

Epochs to convergence

What to notice: train accuracy often rises faster than validation — a small gap is normal. If the gap widens significantly over epochs, that is overfitting.

Reading Learning Curves — Diagnose Training Problems

Underfitting

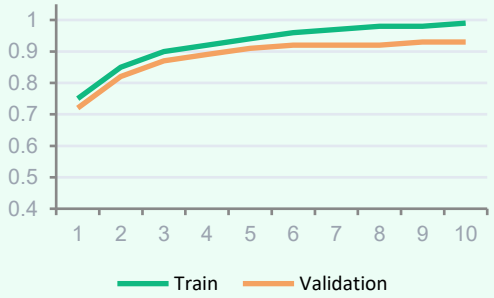
What you see:

Both curves plateau at a low value

What to do:

Add more layers, increase units, train longer, reduce regularization

Good Fit



What you see:

Train ≈ Validation, both approach the target accuracy

What to do:

No immediate action needed — this is the goal. Collect more data or tune for marginal gains.

Overfitting

What you see:

Train accuracy >> Validation; val curve peaks then falls

What to do:

Add Dropout, L2 regularization, data augmentation, or reduce model capacity (L6)

QUICK QUIZ

You train a model for 30 epochs and observe:

- Training accuracy reaches 96 % by epoch 15
- Validation accuracy peaks at 87 % at epoch 11, then slowly declines to 82 % by epoch 30
- Training loss keeps decreasing throughout

Which best describes this situation and the most appropriate next step?

A	Data leakage — the model has seen the test set during training. Re-split the data.
B	Underfitting — the model is too simple. Add more layers and train longer.
C	Overfitting — the model memorizes training data but generalizes poorly. Add regularization or more data.
D	Normal training dynamics — a 9-point train/val gap is expected and no action is needed.

Getting Started — Before the Next Lecture

Lecture 2 · Setup

1 **Run the Fashion MNIST notebook in Colab**

Open Colab → new notebook → Runtime: T4 GPU · Paste code from slide 20 and run · Observe training output each epoch

2 **Experiment — change the architecture**

Change the hidden layer size: try 64 and 256 neurons · Add a second hidden layer: Dense(64, activation='relu') · Change activation in hidden layer to 'tanh' — does accuracy change?

3 **Experiment — read the learning curves**

Train for 20 epochs. Does val accuracy keep improving? · Plot history['accuracy'] vs history['val_accuracy'] · Identify: underfitting / good fit / overfitting

4 **Read before Lecture 3**

Géron Ch. 10: Functional API (pp. 329–335), Callbacks (pp. 338–340) · Focus on: what is the difference between Sequential and Functional API?

Key Takeaways

Perceptron = single linear boundary, cannot learn XOR. MLP = hidden layers + nonlinear activations → model nonlinear relationships.

Forward pass: z = Wx + b then activation f(z), layer by layer. Each neuron's output feeds the next layer.

ReLU default for hidden layers: gradient = 1 when active, no saturation. Sigmoid/Tanh saturate → vanishing gradients in deep nets.

Match loss to task: MSE for regression · Binary CE for 2-class · Sparse Categorical CE for K classes. For standard multiclass classification, use softmax with categorical cross-entropy.

Keras: Sequential([...]) → compile(loss, optimizer, metrics) → fit(X, y, epochs, validation_split) → evaluate(X_test, y_test).

Learning curves: train ≈ val → good fit · both low → underfitting · val peaks then falls → overfitting. Fashion MNIST: ~89% test accuracy after 10 epochs.

Next up: Lecture 3 — Keras API Deep Dive · Functional API · EarlyStopping · ModelCheckpoint